

Lecture 23: Application Layer

Design Patterns, DNS, HTTP, and QUIC

EC 441 — Introduction to Computer Networking
Boston University, Spring 2026

Thursday, April 23, 2026

Learning Objectives

By the end of this lecture you should be able to:

1. Describe the dominant design patterns of application-layer protocols (request/response, stateless vs stateful, text vs binary, push vs pull, short-lived vs persistent connections) and identify the pattern used by any protocol you encounter.
2. Contrast client-server and peer-to-peer architectures, and explain why BitTorrent uses a P2P model while DNS, HTTP, and SMTP use hierarchical or client-server models.
3. Walk through the DNS resolution chain from root through TLD to authoritative servers, and explain how caching and TTLs make the system scale.
4. Narrate the HTTP evolution from HTTP/1.1 → HTTP/2 → HTTP/3 (QUIC), naming the problem each version solves and the problem it leaves for the next version.
5. Explain why email is three protocols, and why browser-based webmail hides most of that complexity from users today.

1 Framing

L22 ended with “*you can now write networked apps from sockets.*” L23 takes the next step: these are the protocols people actually write at that layer.

The application layer is where protocol design has the most freedom — everything above sockets is yours to design. That freedom has produced wildly different choices (text-based HTTP/1.1, binary HTTP/2, fully decentralized BitTorrent, hierarchical DNS), and this lecture walks through how designers have made those choices and why.

All demos use `curl`. This is deliberate: it shows you do not need a browser to drive HTTP. Every feature we discuss — methods, headers, body framing, HTTP/2 streams, HTTP/3 over QUIC — can be exercised from the command line. This reinforces the L22 theme: you have the full stack under your control, GUI or not.

2 Design Patterns

Application-layer protocols sit along a handful of axes:

Axis	Option A	Option B
Interaction	request/response	push / pub-sub
State	stateless	stateful
Encoding	text (ASCII)	binary
Direction	pull	push
Connection	short-lived	persistent
Architecture	client-server	peer-to-peer

Naming new protocols along the axes

When you meet a new application-layer protocol, name it along each axis. You will understand its tradeoffs faster than trying to memorize its details. “Stateless, text, pull, short-lived, client-server” is HTTP/1.1 in five words.

2.1 The stateless + cookie pattern

HTTP is stateless by design: each request stands alone, and the server does not remember you between requests. How does “you are logged in” work, then?

The client carries the state and presents it on every request. The server sets a cookie on the login response; the client stores it and sends it on subsequent requests; the server looks the cookie up in its own session table.

Stateless on the wire. State lives in the cookie jar and in the server’s database. Moving state from “the protocol” to “the client and a database” is the single most important structural decision in modern web architecture — it is what lets a load balancer send your next request to any server in the farm.

2.2 Text vs binary

HTTP/1.1 is ASCII. You can `telnet example.com 80` and type a request by hand. Easy to debug; expensive to parse at scale.

HTTP/2 and HTTP/3 use binary framing. Compact, fast, requires a tool that speaks the wire format to inspect it. A tradeoff worth naming: debuggability vs performance, and the industry has landed on “binary, with good tools.”

2.3 Connection lifetime

- **Short-lived:** DNS queries. One UDP exchange, done.
- **Semi-persistent:** HTTP/1.1 with keep-alive; HTTP/2, HTTP/3 persistent across many requests.
- **Long-lived:** SSH, WebSocket, MQTT — hours or days.

Connection setup is not free (TCP three-way handshake, TLS handshake), so reusing a connection matters. HTTP/2 and HTTP/3 take this to an extreme by multiplexing hundreds of logical streams over one transport connection.

3 Client–Server vs Peer-to-Peer

Two families of organizing principles.

3.1 Client–server: asymmetric roles

The server runs, listens, and waits. Clients initiate. This is the default assumption for almost every protocol we have covered: HTTP, DNS, SSH, SMTP, IMAP.

Scale comes from replication: CDNs (many servers, same content), anycast (same IP, many physical locations), load balancers. Easy to reason about, easy to secure, easy to bill.

3.2 Peer-to-peer: symmetric roles

Every node is both a client and a server. There is no central coordinator, or at most a lightweight one. The canonical example is **BitTorrent**.

BitTorrent as mechanism design

BitTorrent solves the problem of distributing a large file to many recipients without a single server paying the bandwidth cost. Four design beats make it work:

1. **Chunks with cryptographic hashes.** Each chunk of the file has a hash stored in the `.torrent` metadata. You can verify any chunk from any peer — trust is replaced by verification.
2. **Peer discovery.** Trackers (hybrid with a central server) or a distributed hash table (fully decentralized).
3. **Rarest-first chunk selection.** Each peer preferentially downloads chunks that few other peers have. This keeps rare chunks alive in the swarm.
4. **Tit-for-tat choking.** Peers preferentially upload to peers who upload back to them. Free-riders get throttled. This is real game-theoretic mechanism design embedded in a wire protocol.

The architectural win: *popularity increases capacity*. A viral file gets faster, not slower, as more people want it.

3.3 Where P2P fits and where it doesn't

P2P wins when the workload is:

- Bulk file distribution (Linux ISOs, game patches, scientific datasets).
- Censorship-resistant storage (IPFS).
- Global consensus (blockchain).

P2P is a bad fit when the workload needs:

- A single source of truth (account balance, authoritative DNS, email delivery to a specific inbox).

- Low-latency interactive work (chat, video call).
- A clear customer to bill.

3.4 Contrast: DNS is hierarchical, not P2P

DNS is often held up as a “distributed system,” but it is specifically *hierarchical with delegation*, not a P2P mesh. Root servers delegate to TLD servers, which delegate to zone authoritative servers. Each level has a small set of operators; the whole thing is a tree, not a mesh. This is one of the most effective decentralized systems ever built — and it is not P2P.

4 DNS

DNS is the phone book of the internet: it maps names humans use to addresses routers use. It is the largest distributed database in everyday use, and it scales through aggressive caching.

4.1 The namespace

DNS names are a tree, read right to left:

```

www.eng.bu.edu.
|  |  |  |
|  |  |  +-- TLD: edu
|  |  +----- second level: bu
|  +----- subdomain: eng
+----- host: www

```

The trailing `.` is the root. You rarely type it, but every DNS library inserts it for you.

4.2 Resolution chain

When your laptop asks for `www.eng.bu.edu`:

1. The stub resolver on your laptop sends the query to a *recursive resolver* — your ISP, Cloudflare’s 1.1.1.1, Google’s 8.8.8.8, etc.
2. The resolver checks its cache. If hit, done.
3. Otherwise it walks the chain:
 - Root server: “who handles `edu`?”
 - `edu` server: “who handles `bu.edu`?”
 - `bu.edu` server: “who handles `eng.bu.edu`?”
 - `eng.bu.edu` server: “what is the A record for `www.eng.bu.edu`?”
4. Answer bubbles back to your laptop.

Your laptop speaks only to the recursive resolver. The resolver does the walking. This asymmetry — *recursive* at the resolver, *iterative* at the authoritative servers — is the architectural key.

4.3 Record types

Type	Purpose
A	IPv4 address
AAAA	IPv6 address (four A's because it is four times longer)
CNAME	Alias to another name
MX	Mail exchange (with priority)
NS	Name servers for a zone
TXT	Free-form text; SPF, DKIM, domain verification
SOA	Start of authority; zone metadata and TTLs

4.4 Caching and TTLs

Every DNS record carries a TTL set by the authoritative owner. Resolvers (and often clients) cache until the TTL expires.

- Web record TTLs: 300–3600 s. Low TTL gives fast failover at the cost of higher query load.
- Root hint TTLs: about 6 days. They change rarely.
- DNS handles roughly 10^{12} queries per day, on infrastructure that could not possibly answer them all fresh. Caching is what makes the system scale.

dig walkthrough

Script: `demo_dig_trace_123.sh`

```
dig www.eng.bu.edu +short      # just the IP
dig www.eng.bu.edu            # full output
dig @1.1.1.1 www.eng.bu.edu    # specific resolver
dig bu.edu MX +short          # mail exchanges
dig bu.edu NS +short          # name servers
dig +trace www.eng.bu.edu      # walk root -> TLD -> auth
dig -x 8.8.8.8 +short          # reverse lookup
```

In the `+trace` output look for `a.root-servers.net` at the top, followed by the `edu` servers, then `bu.edu`, then the final A record in the `ANSWER` section.

4.5 Transport

Transport	Port	Use
UDP	53	Default. Small queries, small replies.
TCP	53	Large responses, zone transfers, DNSSEC.
TLS	853	DNS over TLS (DoT). Privacy.
HTTPS	443	DNS over HTTPS (DoH). Privacy; blends with web traffic.

DoH is contentious because it looks like HTTPS from the outside. Network operators cannot distinguish it from regular web traffic, which is good for privacy and hard for local policy (enterprise filtering, parental controls, captive portals).

DNSSEC *signs* records so you can verify they are authentic. It does not encrypt queries — that is DoT/DoH’s job. DNSSEC deployment is partial and uneven; it is worth knowing about and not dwelling on.

5 What Is “The Web”? HTML and MIME Types

A common confusion worth resolving before the HTTP deep dive: **HTTP is not the Web**. HTTP fetches one resource at a time — one URL in, one response out. It has no concept of “linked pages,” “embedded images,” or “a website.”

The Web is what emerges when HTML documents reference other resources through links:

- `<a href="...">` — a hyperlink the user can follow
- `` — an image to embed inline
- `<link rel="stylesheet" href="...">` — a CSS file to load
- `<script src="...">` — a JavaScript file to execute
- `<video>`, `<iframe>`, `<object>`, web fonts, tracking pixels, and so on.

The browser as coordinator

HTTP is a dumb fetch protocol. The *browser* is what makes a web page: it parses the HTML, discovers the links to CSS, JavaScript, images, fonts, and embedded content, fires off additional HTTP requests for each of them, executes the JavaScript (which may trigger still more requests), and assembles the final rendered page.

A typical modern page is 1 HTML document plus somewhere between 50 and 500 sub-resources. That workload — many parallel fetches from the same origin — is exactly what HTTP/2 multiplexing and HTTP/3 streams are built to serve. Before we get there, HTTP/1.1 handled it with 6+ parallel TCP connections per origin.

5.1 MIME types

When a browser receives a response, it needs to know *what kind of data* the bytes represent. That is what the **Content-Type** header tells it, using a **MIME type**.

MIME — *Multipurpose Internet Mail Extensions* — was originally designed for email attachments (RFC 2045), but the type system proved so useful that HTTP adopted it, and it is now universal. macOS uses MIME types in the filesystem, package managers use them, and every web API ever written depends on them.

MIME type	Used for
text/html	HTML documents
text/css	Stylesheets
text/plain	Plain text
application/javascript	JavaScript
application/json	JSON (REST APIs, configs)
application/xml	XML
application/pdf	PDF documents
application/octet-stream	Unknown binary (“download me”)
image/png	PNG images
image/jpeg	JPEG images
image/webp, image/avif	Modern image formats
video/mp4, video/webm	Video
audio/mpeg, audio/ogg	Audio

A quick way to see this live:

```
curl -sI https://example.com | grep -i content-type
# Content-Type: text/html; charset=UTF-8

curl -sI https://api.github.com | grep -i content-type
# Content-Type: application/json; charset=utf-8
```

MIME types frequently carry a *parameter* — most often a character set (**charset=UTF-8**) for text content. Browsers and servers use this to decode bytes into characters correctly.

5.2 Why this matters here

The HTML-plus-MIME story explains the shape of modern web traffic and motivates everything in the next three sections:

- HTTP/1.1 with keep-alive was fine for 5–10 sub-resources per page. Modern pages blow past that.
- HTTP/2 exists because browsers were opening 6 TCP connections per origin to parallelize fetches, and that was still not enough.
- HTTP/3 exists because even HTTP/2’s multiplexing suffered from TCP head-of-line blocking when one of those many sub-resource fetches hit packet loss.

With that framing in place, HTTP/1.1 makes sense as a starting point.

6 HTTP/1.1

HTTP/1.1 (RFC 2616, later RFC 9110 for semantics) is the baseline. If you understand this, later versions are additive.

6.1 Request and response

A request:

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: curl/8.4.0
Accept: */*
```

A response:

```
HTTP/1.1 200 OK
Date: Thu, 23 Apr 2026 12:34:56 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 1256
Connection: keep-alive

<!DOCTYPE html>...
```

ASCII all the way down. You can literally `telnet example.com 80` and type a request by hand.

6.2 Key features

- **Methods:** GET, POST, PUT, DELETE, HEAD, OPTIONS, PATCH.
- **Status codes:** 1xx informational, 2xx success, 3xx redirect, 4xx client error, 5xx server error.
- **Headers:** key-value, extensible, case-insensitive.
- **Body framing:** Content-Length or Transfer-Encoding: `chunked`.
- **Stateless:** cookies provide the session illusion.
- **Persistent connections:** Connection: `keep-alive` reuses one TCP connection across many requests.

6.3 HOL blocking at the application layer

HTTP/1.1 pipelining was meant to let a client send multiple requests on one connection without waiting for each response. In practice, pipelining failed because responses must come back *in the order requested*, so one slow response blocks every faster response behind it. Browsers gave up on pipelining and instead opened 6+ parallel TCP connections per origin.

The problem HTTP/2 solves

Application-level head-of-line blocking. One slow response on a pipelined HTTP/1.1 connection holds up every subsequent response. HTTP/2 solves this by multiplexing — many independent streams sharing one TCP connection.

curl with HTTP/1.1

Script: `demo_curl_versions_l23.sh`

```
curl -v --http1.1 https://example.com
```

Verbose output shows the full request and response with every header. Lines starting with `>` are what curl sends; `<` is what the server returns; `*` is curl's own connection state.

This is the point of the demo: *you do not need a browser to drive HTTP*. Every feature above is available from the command line.

7 HTTP/2

HTTP/2 (RFC 7540, 2015) keeps HTTP semantics — same methods, same status codes, same header model — but replaces the wire format.

- **Binary framing.** Every message becomes a sequence of small binary frames. You cannot debug with `telnet` anymore; you need a tool that speaks HTTP/2.
- **Multiplexing over a single TCP connection.** Many streams can be in flight at once; frames from different streams interleave. This solves the application-level HOL blocking that killed HTTP/1.1 pipelining.
- **Header compression (HPACK).** HTTP/1.1 repeats the same headers on every request (User-Agent, cookies, ...). HPACK maintains a shared dictionary and sends only deltas.
- **Server push.** Servers can proactively send resources the client will probably need. Deprecated in practice — browsers found it rarely faster than client-initiated fetches.

The problem HTTP/2 does not solve

TCP-layer head-of-line blocking. Even with 100 multiplexed streams, a single lost TCP segment stalls *all* streams because TCP delivers bytes in order. A missing byte holds back every logical stream behind it. HTTP/3 / QUIC solves this.

curl with HTTP/2

```
curl -v --http2 -o /dev/null https://www.cloudflare.com
```

Verbose output shows using HTTP/2 and HTTP/2 pseudo-headers (`:method`, `:path`, `:authority`) — what the HTTP/1 method, path, and Host header became in the binary wire format.

8 HTTP/3 and QUIC

HTTP/3 (RFC 9114, 2022) runs HTTP over **QUIC** (RFC 9000), which is a reliable transport protocol built on top of **UDP**.

8.1 Why UDP

TCP cannot be meaningfully changed in the kernel of every router, OS, and middlebox on the internet. Middleboxes — NATs, firewalls, load balancers — make strong assumptions about TCP and break when it changes. UDP is dumb enough that middleboxes mostly leave it alone. QUIC moves the reliable transport logic into *user space*, built on UDP. A browser update can now ship a new transport to a billion users overnight, without a kernel patch.

8.2 What QUIC provides

- **Reliable, ordered delivery per stream** — like TCP, but per-stream.
- **Multiplexed independent streams.** A lost packet affects only the stream it was in; other streams keep moving. TCP HOL gone.
- **Built-in TLS 1.3.** Encryption is not a layer on top; it is part of the handshake. One round-trip establishes both the transport connection and the cryptographic session.
- **0-RTT resumption.** When reconnecting to a server you have spoken to before, you can send application data in the first packet.
- **Connection migration.** A QUIC connection ID is not tied to the IP/port tuple; a client moving from WiFi to cellular keeps the same connection.

Callbacks to earlier lectures

L18: UDP was introduced alongside the teaser that QUIC is why UDP has a second act. This is it.

L20: QUIC runs its own congestion control — often CUBIC or BBR — in user space. Same algorithms, new implementation surface.

L22: QUIC uses a UDP socket. Everything above the socket is library code.

curl with HTTP/3

```
curl -v --http3 -o /dev/null https://www.cloudflare.com
```

Requires a curl built with HTTP/3 support. Check with `curl --version` — the Features line should list HTTP3. On macOS, `brew install curl` gives a modern build; the system curl may not include HTTP/3.

8.3 Timeline

Year	Version	Transport	Wire	Main improvement
1996	HTTP/1.0	TCP	text	—
1999	HTTP/1.1	TCP	text	keep-alive, Host header
2015	HTTP/2	TCP	binary	multiplexing, HPACK
2022	HTTP/3	QUIC (UDP)	binary	TCP HOL gone, TLS built-in

HTTP *semantics* (methods, status codes, headers) have barely changed. The wire format, transport, and connection model keep evolving.

9 Email and the Webmail Complication

Email is *three* protocols:

- **SMTP** (ports 25 / 587 / 465): sending. Client-to-server submission and server-to-server relay.
- **IMAP** (ports 143 / 993): reading mailboxes on a server.
- **POP3** (ports 110 / 995): downloading mailboxes; largely legacy.

Classical picture: a desktop mail client speaks SMTP to send and IMAP to read. Every user ran a full mail client.

The browser complication

Gmail, Outlook.com, and almost every modern webmail are *not* speaking SMTP or IMAP to your browser. Your browser speaks **HTTPS** to a Google web application. Google's servers speak SMTP and IMAP *on your behalf* inside their infrastructure — to Google's own mail storage, and to external mail servers (outlook.com, bu.edu) for delivery across providers. The user-facing protocol is JSON-over-HTTPS, not email.

Consequence: most users no longer touch SMTP or IMAP directly. The protocols are still there — servers still relay mail over SMTP — but they are hidden from end users by an HTTPS web app. This is one reason email has been slow to evolve: the public protocol surface is large, different webmail providers ship different UI over it, and the wire protocols between servers face strong backward-compatibility pressure.

10 WebSocket: the Escape Hatch from Request/Response

Some applications — chat, live dashboards, collaborative editing, multiplayer games — do not fit request/response. The client wants to be *notified when things happen*, not have to poll.

WebSocket (RFC 6455) solves this. It starts with an HTTP request that *upgrades* the connection:

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: ...
```

After a 101 **Switching Protocols** response, the connection stops being HTTP and becomes a persistent, bidirectional, full-duplex frame channel. Both ends can send messages at any time.

WebSocket is ubiquitous in modern web apps — Slack, Google Docs, live stock feeds, Discord all use it. It is the canonical example of HTTP being extended past its original request/response model.

11 Full Circle: Back to Lecture 2

Before the bridge to L24, a moment to close a much bigger loop.

Three questions worth asking about every modern web page:

- Why is the string "Hello <two Chinese chars>" in an HTML document only 12 bytes on the wire, not 24?
- Why is a 2-megapixel photo typically a 100 KB JPEG, not 6 MB?
- Why is 1080p Netflix only about 5 Mb/s, not 1.5 Gb/s?

The answer to all three is Lecture 2 — information theory, entropy, and encoding. Every MIME type you saw earlier carries an encoding whose job is to push the byte count toward the Shannon lower bound.

Content	MIME type	What L2 said
HTML text	<code>text/html; charset=UTF-8</code>	UTF-8: ASCII compat, 1–4 B/char
Stylesheet	<code>text/css</code>	gzip/Brotli in transit
JavaScript	<code>application/javascript</code>	gzip/Brotli in transit
Photograph	<code>image/jpeg</code>	6.2 MB → 100 KB ($\sim 60\times$)
Video stream	<code>video/mp4</code>	1.49 Gb/s → 5 Mb/s ($\sim 300\times$)

Without Lecture 2, none of this works

An uncompressed 1080p video stream at 30 fps would need 1.49 Gb/s per viewer — saturating a 1 Gb/s Ethernet link for a single stream. The global video traffic we casually consume every day only exists because of the compression theory and algorithms introduced in L2. *Every byte on the wire was put there by someone who read Shannon.*

11.1 One HTTP response, every layer

When you load a single frame of a Netflix stream, here is what each part of the course contributes:

Layer and lecture	What it contributes
L2 — information	H.264 encodes a frame: ~ 1.49 Gb/s → 5 Mb/s
L3 — physical	Bits on fiber, copper, or radio
L4 — wireless	OFDM subcarriers on 802.11ax
L5–L6 — error coding	CRC and FEC protect each frame
L7 — MAC	CSMA/CA arbitrates the air
L8 — Ethernet	Frame with preamble, MAC, FCS
L9 — WiFi	802.11 frame, probe/auth/associate done
L13–L17 — network	IP datagram, TTL, routing, NAT
L18–L20 — transport	TCP (or QUIC) segment, cwnd, RTT
L21–L22 — tools	<code>ss</code> , <code>tcpdump</code> , <code>sockets</code> , <code>Scapy</code> , <code>Mininet</code>
L23 — application	HTTP/2 stream carrying <code>video/mp4</code>
→ screen	One decoded 1080p frame. 33 ms later, the next.

The whole semester lives in one HTTP response.

12 Bridge to L24

Every protocol in today's lecture runs in *cleartext* on the wire by default:

- HTTP/1.1 headers, cookies, and bodies are visible to anyone on the path.
- DNS queries are visible to your ISP, your WiFi access point's owner, and any network operator between you and the resolver.
- SMTP between servers is often encrypted via STARTTLS, but was not originally.

QUIC and DoH are early wins in pushing encryption into the default. But the big answer is **HTTPS**, which is HTTP over **TLS**. TLS is built from the cryptography we will cover in L24: symmetric ciphers, asymmetric ciphers, hash functions, digital signatures, and a global PKI of certificates.

Everything you watched today, tcpdump could read. Next lecture: how we fix that.

References

- Wikipedia: Application layer
- Wikipedia: Domain Name System
- Wikipedia: Hypertext Transfer Protocol
- Wikipedia: HTTP/2
- Wikipedia: HTTP/3
- Wikipedia: QUIC
- Wikipedia: BitTorrent
- Wikipedia: WebSocket
- Wikipedia: SMTP
- RFC 9110 — HTTP Semantics
- RFC 9114 — HTTP/3
- RFC 9000 — QUIC
- RFC 1035 — DNS (still current)
- curl manual